

Construindo o “R” do RAG: Um Sistema de Recuperação de Artigos Científicos sobre Engenharia de Software Cross-Platform

Diego Rodrigues de Souza Arimura¹

¹Faculdade de Computação (FACOM)
Fundação Universidade Federal de Mato Grosso do Sul (UFMS)
Campo Grande – MS – Brasil

`diego.arimura@ufms.br`

Abstract. Retrieval-Augmented Generation (RAG) has emerged as a cornerstone for building reliable question-answering systems over specific knowledge bases. In this context, the retrieval component (the “R”) is responsible for selecting the most relevant documents to feed the generative model. This work designs, implements, and evaluates a document retriever on a scientific paper collection from ArXiv containing 2,699 documents related to cross-platform mobile software engineering (Flutter, React Native, Android, and iOS). We compare three retrieval strategies: a lexical sparse baseline (BM25), a dense semantic retriever (KNN using all-MiniLM-L6-v2 embeddings), and a hybrid retriever utilizing Reciprocal Rank Fusion (RRF). Evaluating these models over 15 targeted queries shows that the hybrid approach balances the strengths of both representations, achieving the best overall Recall@10 (0.5685) and Top Precision@10 (0.5333).

Resumo. A Geração Aumentada de Recuperação (RAG) consolidou-se como uma abordagem fundamental para construir sistemas confiáveis de perguntas e respostas sobre bases de conhecimento específicas. Nesse contexto, o componente de recuperação (o “R”) é responsável por selecionar os documentos mais relevantes para alimentar o modelo gerador. Este trabalho projeta, implementa e avalia um recuperador de documentos sobre uma coleção de 2.699 artigos científicos extraídos do ArXiv relacionados à engenharia de software móvel cross-platform (Flutter, React Native, Android e iOS). Comparamos três estratégias: um baseline esparsa léxico (BM25), um recuperador denso semântico (KNN usando embeddings all-MiniLM-L6-v2) e um recuperador híbrido utilizando Reciprocal Rank Fusion (RRF). A avaliação dos modelos sobre 15 queries direcionadas demonstra que a abordagem híbrida balanceia as forças de ambas as representações, obtendo a melhor cobertura Recall@10 (0.5685) e Precisão@10 de topo (0.5333).

1. Introdução

A produção literária na área de tecnologia da informação tem crescido a taxas exponenciais, criando um desafio de sobrecarga cognitiva para pesquisadores e profissionais no momento de identificar trabalhos relevantes. No domínio do desenvolvimento móvel, o advento de tecnologias de desenvolvimento híbrido e multiplataforma (*cross-platform*),

como Flutter (baseado na linguagem Dart) e React Native (baseado em JavaScript/TypeScript), gerou uma grande quantidade de literatura acadêmica enfocando tópicos como testes automatizados, consumo energético, qualidade de código e desempenho em relação às plataformas nativas tradicionais (Android/Java/Kotlin e iOS/Swift).

Em paralelo, a ascensão dos Grandes Modelos de Linguagem (LLMs - *Large Language Models*) proporcionou novas capacidades de interação em linguagem natural com bases de dados e repositórios de código. Contudo, esses modelos sofrem com limitações inerentes de conhecimento temporal ou privado, além da propensão a alucinações (geração de fatos falsos mas textualmente coerentes). Para contornar esse desafio, a arquitetura de Geração Aumentada por Recuperação (RAG - *Retrieval-Augmented Generation*) consolidou-se como o padrão da indústria. Em um sistema RAG, antes que a fase generativa baseada em LLM seja iniciada, um componente de busca — o “R” — atua sobre um corpus pré-indexado para extrair um conjunto conciso de documentos semanticamente relevantes à consulta. Caso este componente falhe e entregue documentos ruidosos ou irrelevantes ao modelo gerador, a resposta final gerada decairá severamente, conforme o clássico princípio da computação *garbage in, garbage out*.

Este trabalho aborda o desenvolvimento, a avaliação rigorosa e a análise conceitual de um sistema de recuperação de artigos científicos do ArXiv focado em Engenharia de Software Móvel Cross-Platform. Investigamos os limites e as forças de abordagens léxicas tradicionais baseadas em termos exatos (BM25 Okapi), técnicas de recuperação semântica densa baseadas em embeddings de aprendizado profundo (KNN com vetores densos de baixa dimensionalidade) e estratégias híbridas que fundem os dois paradigmas utilizando o algoritmo *Reciprocal Rank Fusion* (RRF).

2. Trabalhos Relacionados

A evolução dos sistemas de Recuperação de Informação (IR - *Information Retrieval*) é tradicionalmente dividida entre abordagens baseadas em correspondência de termos (léxicas esparsas) e abordagens baseadas em significado conceitual (semânticas densas).

Recuperação Léxica Esparsa: A indexação léxica esparsa é historicamente liderada pela clássica representação vetorial TF-IDF (Frequência do Termo - Frequência Inversa do Documento) descrita por Manning et al. (2008), e consolidada pelo modelo estatístico BM25 Okapi proposto por Robertson e Zaragoza (2009). Esses modelos focam na correspondência exata de palavras-chave, tratando o texto como um saco de palavras (*bag-of-words*). Embora o BM25 seja computacionalmente eficiente e possua alta precisão para palavras-chave muito específicas e raras (como códigos de erro ou nomes de frameworks), ele sofre com o problema da incompatibilidade de vocabulário (*vocabulary mismatch*). Se um usuário realiza uma consulta utilizando o termo “testes de interface” e um artigo científico relevante utiliza o termo “widget testing” ou “UI evaluation”, o BM25 não computará similaridade devido à falta de correspondência exata dos caracteres, gerando lacunas de busca.

Recuperação Semântica Densa: Com os avanços em redes neurais profundas de processamento de linguagem natural (PLN), modelos como o *Dense Passage Retrieval* (DPR) proposto por Karpukhin et al. (2020) demonstraram a eficácia de codificar consultas e passagens em espaços vetoriais contínuos de tamanho fixo. Em vez de representar textos em vetores esparsos do tamanho do vocabulário total da coleção, esses modelos

utilizam arquiteturas baseadas em Transformers (como BERT) para mapear o texto para vetores densos (normalmente entre 384 e 768 dimensões). Adicionalmente, Reimers e Gurevych (2019) propuseram a arquitetura Sentence-BERT (SBERT), utilizando redes siamesas para treinar o modelo especificamente na geração de embeddings de sentenças e parágrafos semanticamente coerentes, em que a distância cosseno reflete a proximidade de significado conceitual. No domínio científico, Cohan et al. (2020) apresentaram o SPECTER, um codificador treinado por meio de relações de citação entre artigos científicos, provando que embeddings focados no jargão acadêmico superam consideravelmente os codificadores genéricos.

Abordagens Híbridas e Fusão de Ranks: Mais recentemente, a literatura de IR convergiu para sistemas de busca híbridos para contornar as deficiências de cada representação isolada. Conforme discutido por Luan et al. (2021), os modelos esparsos garantem a presença de termos técnicos exatos, enquanto os modelos densos capturam a intenção do usuário e sinônimos contextuais. Para combinar essas saídas sem a necessidade de calibrar pesos de pontuações de naturezas distintas (já que a escala do BM25 é ilimitada e a similaridade de cosseno varia entre -1 e 1), Cormack et al. (2009) introduziram o algoritmo *Reciprocal Rank Fusion* (RRF). O RRF provou ser um método de fusão sem parâmetros altamente robusto e estável, superando abordagens estatísticas de normalização padrão de scores. Por essa razão, a fusão baseada em RRF foi adotada como módulo de aprofundamento e avaliação neste trabalho.

3. Metodologia

O fluxo de trabalho foi construído em cinco etapas integradas: definição do escopo da coleção, coleta via API, normalização e pré-processamento, indexação esparsa, codificação densa e fusão híbrida.

3.1. Escopo da Coleção e Relação com o Projeto de Pesquisa

Esta pesquisa se insere diretamente no contexto de investigação acadêmica e profissional do aluno Diego Rodrigues de Souza Arimura no programa de pós-graduação da Faculdade de Computação (FACOM) da UFMS. O aluno foca seus estudos na área de Engenharia de Software Aplicada a Dispositivos Móveis, com ênfase na identificação de padrões de projeto (*design patterns*), testes automatizados (unidade, widget e integração) e análise de desempenho comparativo entre abordagens cross-platform e nativas.

A construção de um sistema de recuperação de artigos acadêmicos serve como ferramenta de suporte sistemático à pesquisa. Em vez de realizar buscas manuais em bases científicas fragmentadas, o estudante pode utilizar o componente de busca desenvolvido para construir um pipeline RAG local. Esse pipeline é capaz de recuperar evidências científicas concretas para responder a consultas complexas sobre boas práticas de engenharia de software para Flutter e React Native, agilizando revisões sistemáticas de literatura e a fundamentação teórica de sua dissertação de mestrado.

3.2. Coleta e Construção do Corpus

A coleção de artigos científicos foi extraída programaticamente por meio da API do ArXiv, utilizando uma estratégia de busca direcionada para Engenharia de Software Móvel. Filtramos as submissões dentro das seguintes taxonomias de categorias de Ciência da

Computação: `cs.SE` (Software Engineering - qualidade, arquitetura e testes), `cs.HC` (Human-Computer Interaction - usabilidade), `cs.PL` (Programming Languages - paradigmas de linguagens), `cs.CY` (Computers and Society - adoção e impacto) e `cs.CR` (Cryptography and Security - segurança).

O critério de coleta aplicou a seguinte query lógica unificada para os campos de título e abstract (ajustada para não exceder as margens da página):

```
(all:"flutter" OR all:"react native" OR all:"cross-platform" OR
all:"mobile app" OR all:"mobile application" OR
all:"mobile development" OR all:"mobile software" OR
all:"dart programming" OR all:"android" OR all:"ios")
AND (cat:cs.SE OR cat:cs.HC OR cat:cs.PL OR cat:cs.CY OR cat:cs.CR)
```

A janela temporal foi delimitada de 2015 a 2026. A coleta paginada da API do ArXiv com tratamento de erros HTTP 429 e 503 e *backoff* exponencial resultou na coleta de 2.699 artigos. Após a remoção de duplicatas por `arxiv_id` e registros com abstracts vazios ou muito curtos, a base limpa de documentos foi consolidada no formato JSON Lines em `data/corpus.jsonl`, contendo exatamente 2.699 artigos científicos.

3.3. Pré-processamento Textual

Para cada documento, o texto indexável foi gerado pela concatenação de título e abstract. O pipeline de pré-processamento executado em `src/preprocessing.py` realiza sequencialmente: (1) **Normalização** (conversão para minúsculas); (2) **Tokenização** (uso da expressão regular `r"\b[a-zA-Z][a-zA-Z\-\_]+,\b"` para preservar termos com hífen, como “cross-platform” ou “ci-cd”); e (3) **Remoção de Stopwords** (eliminação de palavras vazias em inglês com a biblioteca NLTK, descrita por Bird et al. 2009).

Para fins didáticos, a Tabela 1 ilustra a evolução de uma sentença típica de entrada através do pipeline de pré-processamento.

Tabela 1. Exemplo do fluxo de processamento de texto para um artigo típico.	
Estágio	Conteúdo do Texto
Entrada Bruta	“Automated Testing of Flutter Mobile Applications. In this paper, we discuss unit and integration testing in Flutter.”
Normalização	“automated testing of flutter mobile applications. in this paper, we discuss unit and integration testing in flutter.”
Tokenização Regex	["automated", "testing", "of", "flutter", "mobile", "applications", "in", "this", "paper", "we", "discuss", "unit", "and", "integration", "testing", "in", "flutter"]
Remoção de Stopwords	["automated", "testing", "flutter", "mobile", "applications", "paper", "discuss", "unit", "integration", "testing", "flutter"]

Optou-se por não aplicar técnicas de radicalização (*stemming*) ou lematização. Essa decisão é justificada para preservar a integridade de termos tecnológicos específicos

(ex: diferenciar “Flutter” e “Fluttering” ou “Dart” e “Darting”), minimizando colisões semânticas em termos que possuem significados completamente diferentes na computação em relação à língua inglesa comum.

3.4. Recuperação Esparsa - BM25

O BM25 Okapi atribui pontuações aos documentos com base no alinhamento de termos da consulta no corpus. A pontuação de um documento D para uma query $Q = \{q_1, q_2, \dots, q_n\}$ é dada por:

$$\text{Score}_{BM25}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)} \quad (1)$$

onde $f(q_i, D)$ é a frequência do termo q_i no documento D , $|D|$ é o tamanho do documento em tokens, avgdl é o comprimento médio de documentos do corpus, k_1 é o hiperparâmetro de saturação do termo (definido como 1.5) e b é o fator de penalização de tamanho do documento (definido como 0.75). O cálculo do IDF é modelado como:

$$\text{IDF}(q_i) = \ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right) \quad (2)$$

onde N é o número total de documentos no corpus e $n(q_i)$ é o número de documentos que contêm o termo q_i .

3.5. Recuperação Densa - KNN com Embeddings

Para superar as barreiras de vocabulário, o recuperador semântico denso codifica as sentenças em vetores de números reais. Adotamos o modelo de rede neural pré-treinado `all-MiniLM-L6-v2` do framework `SentenceTransformers`. Este modelo mapeia o texto de título + resumo para um espaço densamente povoado de $d = 384$ dimensões.

Cada documento D_j é representado por um vetor normalizado $\mathbf{v}_{d_j} \in \mathbb{R}^{384}$ tal que $\|\mathbf{v}_{d_j}\|_2 = 1$. Os embeddings de todos os 2.699 documentos são pré-computados e salvos localmente em um arquivo binário do NumPy (`doc_embeddings_all-MiniLM-L6-v2.npy`), permitindo carregamento rápido.

No momento da busca, a query Q é processada pela mesma rede neural, gerando o vetor da query normalizado $\mathbf{v}_q \in \mathbb{R}^{384}$. O cálculo da similaridade semântica para cada documento é computado via produto interno (similaridade de cosseno):

$$\text{Score}_{KNN}(D_j, Q) = \mathbf{v}_q \cdot \mathbf{v}_{d_j} = \sum_{i=1}^{384} v_{q,i} \cdot v_{d_j,i} \quad (3)$$

Utilizamos multiplicação matricial nativa do NumPy para projetar os scores simultaneamente em toda a coleção, selecionando os K documentos de maior similaridade.

Cabe discutir a escolha da similaridade de cosseno frente à distância euclidiana (L_2) tradicional vista em sala para o algoritmo KNN. Como os vetores de embeddings gerados pelo `SentenceTransformers` são previamente normalizados para comprimento unitário ($\|\mathbf{v}\|_2 = 1$), a similaridade de cosseno é linearmente correlacionada com a distância euclidiana ao quadrado: $\|\mathbf{v}_q - \mathbf{v}_d\|_2^2 = 2 - 2\text{sim}_{\cos}(\mathbf{v}_q, \mathbf{v}_d)$. Optou-se pela similaridade de cosseno por ser uma medida padrão na literatura de IR, capturando puramente a orientação dos vetores no espaço multidimensional. Isso evita distorções caso

houvesse variação na magnitude decorrente de diferentes tamanhos de textos, sendo mais estável em comparação com a distância euclidiana pura sobre passagens de texto.

3.6. Ranking Híbrido por Reciprocal Rank Fusion (RRF)

Como módulo de aprofundamento obrigatório para nível de Mestrado, implementamos o algoritmo *Reciprocal Rank Fusion* (RRF). O RRF funde as saídas independentes dos retrievers léxico (BM25) e semântico (KNN) sem a necessidade de normalização prévia de scores de escalas diferentes. A pontuação RRF para cada documento d do pool de candidatos é calculada por:

$$\text{RRF_Score}(d) = \sum_{m \in M} \frac{1}{c + \text{rank}_m(d)} \quad (4)$$

onde $M = \{BM25, KNN\}$ é o conjunto de modelos, $\text{rank}_m(d)$ é a posição de rank ordinal do documento d na lista ordenada pelo modelo m (sendo 1 para o primeiro documento recuperado). A constante de suavização c é definida como 60, valor padrão consolidado na literatura de IR para suavizar a influência de posições de baixa classificação. Caso um documento não seja recuperado por um dos modelos, sua posição rank é considerada infinita, resultando em $\frac{1}{\infty} = 0$ na soma do respectivo modelo.

4. Avaliação Experimental

4.1. Conjunto de Consultas de Teste

Definimos 15 queries de avaliação específicas que cobrem diferentes vertentes do tema de engenharia de software e desenvolvimento móvel, detalhadas na Tabela 2.

Tabela 2. Conjunto de queries de teste utilizadas para a avaliação do retriever.

ID	Query Textual
q1	automated testing of flutter mobile applications
q2	code smells and refactoring in dart projects
q3	performance optimization techniques for cross-platform mobile apps
q4	memory leaks detection in flutter and react native
q5	state management patterns in flutter applications
q6	continuous integration and delivery pipelines for mobile apps
q7	energy consumption and battery efficiency of mobile software
q8	widget testing and integration testing in flutter
q9	static code analysis tools for dart programming language
q10	user interface design and usability testing for mobile applications
q11	cross-platform vs native mobile app development performance
q12	architectural patterns in dart and flutter projects
q13	security vulnerabilities in cross-platform mobile frameworks
q14	automated bug detection and error logging in mobile apps
q15	dart compilation and compile-time optimization in flutter

4.2. Pooling e Relevance Judgments (qrels)

Para viabilizar o cálculo das métricas de recuperação sem precisar avaliar manualmente todos os 2.699 documentos para cada uma das 15 queries (um esforço inviável de $15 \times$

2.699 = 40.485 anotações), aplicamos a técnica tradicional de *pooling*. Para cada query, coletamos os 10 primeiros documentos retornados pelo BM25, KNN e Híbrido, unimos estes resultados e removemos duplicatas. Isso gerou um conjunto de **276 pares documento-query únicos** para avaliação manual. Cada par foi anotado no gabarito `eval/qrels.tsv` com o script `src/annotate_qrels.py`, aplicando as seguintes diretrizes graduadas de relevância: **Relevância 2** (Altamente relevante, abordando diretamente o tema da consulta, como propostas de testes Flutter para q1); **Relevância 1** (Relevante de forma geral, abordando o domínio amplo sem focar na tecnologia exata); e **Relevância 0** (Não-relevante para o escopo da query). O gabarito final foi persistido no arquivo `eval/qrels.tsv`.

4.3. Métricas de Avaliação

Avaliamos as runs geradas com $k = 10$ utilizando o script `eval/evaluate.py`. As métricas calculadas foram:

Precisão@k (P@k): Proporção de documentos relevantes no topo k :

$$P@k = \frac{\sum_{i=1}^k \mathbb{I}(\text{relevância}(d_i) > 0)}{k} \quad (5)$$

Recall@k (R@k): Proporção de documentos relevantes recuperados no topo k em relação ao total de relevantes conhecidos na tabela `qrels` (R_{total}):

$$R@k = \frac{\sum_{i=1}^k \mathbb{I}(\text{relevância}(d_i) > 0)}{R_{total}} \quad (6)$$

Average Precision (AP): Média das precisões calculadas nos pontos onde um documento relevante foi retornado:

$$AP = \frac{1}{R_{total}} \sum_{i=1}^{\text{len}(run)} P@i \cdot \mathbb{I}(\text{relevância}(d_i) > 0) \quad (7)$$

O MAP (*Mean Average Precision*) é a média aritmética das APs calculadas sobre todas as 15 queries de teste.

Normalized Discounted Cumulative Gain (nDCG@k): Medida de ganho acumulado que penaliza documentos relevantes posicionados tardiamente no rank, normalizado pelo ganho ideal acumulado (IDCG):

$$DCG@k = \sum_{i=1}^k \frac{\text{relevância}(d_i)}{\log_2(i+1)}, \quad nDCG@k = \frac{DCG@k}{IDCG@k} \quad (8)$$

Apresentamos um exemplo prático para uma query hipotética com $R_{total} = 4$ relevantes no gabarito, retornando a lista de 5 documentos ($k = 5$) com relevâncias graduadas $[2, 0, 1, 0, 2]$ (onde relevância > 0 indica relevante). Logo, temos: 3 documentos relevantes no topo 5 (D_1, D_3, D_5).

Precisão@5 e Recall@5: $P@5 = \frac{3}{5} = 0.6000$ (60%) e $R@5 = \frac{3}{4} = 0.7500$ (75%).

Average Precision (AP): Calculamos as precisões nos ranks dos documentos relevantes (ranks 1, 3 e 5), obtendo $P@1 = \frac{1}{1} = 1.0$, $P@3 = \frac{2}{3} \approx 0.6667$ e $P@5 = \frac{3}{5} = 0.60$. A média das precisões sobre o total de relevantes conhecidos ($R_{total} = 4$) resulta em: $AP = \frac{1.0+0.6667+0.60}{4} \approx 0.5667$.

nDCG@5: O ganho acumulado descontado obtido é $DCG@5 = \frac{2}{\log_2 2} + \frac{0}{\log_2 3} + \frac{1}{\log_2 4} + \frac{0}{\log_2 5} + \frac{2}{\log_2 6} = 2 + 0.5 + 0.7737 = 3.2737$. Ordenando as relevâncias graduadas do gabarito para obter o topo ideal $([2, 2, 1, 0, 0])$, computamos $IDCG@5 = \frac{2}{\log_2 2} + \frac{2}{\log_2 3} + \frac{1}{\log_2 4} = 2 + 1.2618 + 0.5 = 3.7618$. A normalização resulta em: $nDCG@5 = \frac{3.2737}{3.7618} \approx 0.8702$.

5. Resultados e Discussão

5.1. Análise Quantitativa por Consulta

Os três modelos foram avaliados em relação às 15 queries de teste para a configuração $k = 10$. Os resultados exatos por query estão detalhados na Tabela 3.

Tabela 3. Resultados detalhados por consulta para os modelos esparsos (BM25), denso (KNN) e híbrido ($k = 10$).

QID	BM25 (Esparsos)				KNN (Denso)				Híbrido (RRF)			
	P@10	R@10	AP	nDCG@10	P@10	R@10	AP	nDCG@10	P@10	R@10	AP	nDCG@10
q1	1.0000	0.5000	0.6782	0.9785	1.0000	0.5000	0.6928	0.6807	1.0000	0.5000	0.8462	0.8225
q2	0.6000	0.8571	0.8095	0.8932	0.6000	0.8571	0.8766	0.8848	0.6000	0.8571	0.8690	0.8972
q3	1.0000	0.5263	0.6774	0.8765	0.9000	0.4737	0.5083	0.6392	0.8000	0.4211	0.6850	0.6725
q4	0.4000	0.5714	0.4003	0.6208	0.2000	0.2857	0.1423	0.1493	0.4000	0.5714	0.3782	0.5650
q5	0.0000	0.0000	0.0909	0.0000	0.0000	0.0000	0.0167	0.0000	0.1000	1.0000	0.1000	0.2891
q6	0.9000	0.9000	0.9414	0.9205	0.5000	0.5000	0.4861	0.6711	0.7000	0.7000	0.7310	0.7975
q7	1.0000	0.6250	0.9453	1.0000	1.0000	0.6250	0.9188	1.0000	1.0000	0.6250	0.9708	1.0000
q8	0.1000	0.3333	0.1792	0.1480	0.1000	0.3333	0.1764	0.2021	0.3000	1.0000	0.3333	0.5431
q9	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
q10	1.0000	0.7143	0.9167	0.9682	0.7000	0.5000	0.6065	0.6157	1.0000	0.7143	0.9470	0.9302
q11	0.2000	0.4000	0.2115	0.2614	0.4000	0.8000	0.3887	0.5117	0.3000	0.6000	0.3630	0.3765
q12	0.0000	0.0000	0.0384	0.0000	0.3000	1.0000	0.2296	0.4441	0.1000	0.3333	0.1645	0.2021
q13	0.5000	0.3846	0.4286	0.4825	0.6000	0.4615	0.3814	0.4944	0.7000	0.5385	0.5535	0.5280
q14	1.0000	0.6667	0.8550	1.0000	1.0000	0.6667	0.7650	0.7790	1.0000	0.6667	0.9150	0.8923
q15	0.1000	1.0000	1.0000	1.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0400	0.0000
Média	0.5200	0.4986	0.5448	0.6100	0.4867	0.4669	0.4126	0.4715	0.5333	0.5685	0.5264	0.5677

A análise geral das médias na Tabela 3 indica que o sistema **Híbrido (RRF)** atinge a melhor cobertura geral, obtendo o maior Recall@10 médio (**0.5685**) e a melhor Precisão de topo (**P@10 = 0.5333**). A fusão baseada em classificação recíproca obteve sucesso ao balancear o melhor de dois mundos: os termos exatos de alta pontuação IDF do BM25 com as relações contextuais trazidas pela proximidade vetorial do KNN. Por sua vez, o **BM25** individual apresentou a maior pontuação de MAP geral (0.5448) e nDCG@10 (0.6100). Isso ocorre devido à natureza altamente técnica e de domínio específico das consultas de Engenharia de Software. Em termos práticos, consultas contendo palavras como “linter”, “smells” ou “pipelines” retornam poucos resultados que coincidem no corpus. A correspondência de termos exatos atua como um filtro rigoroso, e quando os termos se alinham perfeitamente, o **BM25** posiciona o documento correto no rank 1, maximizando o ganho acumulado descontado.

O modelo **KNN Denso**, isoladamente, obteve resultados discretamente menores em relação ao MAP (0.4126). Isso se deve a um efeito conhecido da representação em vetores de tamanho fixo de modelos pré-treinados em domínio amplo. O modelo de embeddings `all-MiniLM-L6-v2` ocasionalmente introduz falsos positivos conceituais, ou seja, documentos que discutem conceitos vagamente semelhantes (como nuvens ou containerização) mas sem relação direta com a consulta específica de desenvolvimento de apps móveis. Em contrapartida, o **KNN** é muito mais estável, nunca zerando o nDCG em consultas difíceis (como na query q12), onde o BM25 falhou totalmente.

No **Caso 1** (Query q_5 — “*state management patterns in flutter applications*”), o BM25 e o KNN recuperaram no topo o artigo relevante ID 2407.19318 (ASM). No entanto, o BM25 inseriu no rank 3 o ruído léxico ID 2309.12365 (um sistema de estoque de armazém) devido à coincidência de termos comuns como “management” e “system”. Em contraste, o KNN recuperou no rank 2 o artigo ID 2502.11708 (Docker Controller App em Flutter), mantendo maior proximidade semântica ao contexto de programação móvel.

No **Caso 2** (Query q_{12} — “*architectural patterns in dart and flutter projects*”), o BM25 obteve precisão zero, trazendo artigos de saúde contendo o termo “architectural” isolado. Já o KNN obteve sucesso ao recuperar no rank 2 o artigo ID 1610.05711 (sobre anti-padrões e qualidade em Android). Mesmo sem conter o termo literal “architectural patterns”, o modelo semântico associou o conceito com “anti-patterns” e “code smells”, superando a incompatibilidade de vocabulário da busca léxica.

6. Conexões com a Disciplina de IA

Os experimentos realizados ilustram de forma prática conceitos teóricos fundamentais estudados na disciplina:

Modelo Probabilístico de IR e Naïve Bayes: A formulação de relevância do BM25 mapeia-se diretamente ao modelo probabilístico Naïve Bayes aplicado a texto. Ambos os paradigmas assumem a hipótese de independência de termos no documento (saco de palavras), modelando a relevância como a soma de probabilidades condicionais independentes de cada palavra-chave dada a classe (relevante/não-relevante).

Espaço Vetorial, Distâncias e o Algoritmo KNN: A busca densa reflete o algoritmo clássico de busca por vizinhos mais próximos (K -Nearest Neighbors) aplicado a tarefas de recomendação. Em vez de usar os vizinhos para classificação discreta via votação majoritária, o retriever ordena os vizinhos mais próximos no espaço vetorial multidimensional (384 dimensões) com base na similaridade de cosseno.

Redes Neurais e Aprendizado de Representações: A geração dos embeddings baseia-se na rede neural SentenceTransformer (arquitetura Transformer). O treinamento desta rede foi feito via aprendizado contrastivo, no qual o modelo ajusta seus pesos internos através de retropropagação de erro (*backpropagation*) para aproximar sentenças semanticamente equivalentes e afastar dissemelhantes.

7. Declaração de Uso de IA Generativa

Declara-se o uso assistido do assistente inteligente Antigravity (Google DeepMind) exclusivamente para apoio na formatação de tabelas LaTeX, revisão gramatical do artigo e sugestão de trechos do script Flask. Toda a análise conceitual, anotações de relevância e decisões de projeto foram de autoria intelectual do próprio autor.

8. Conclusão e Trabalhos Futuros

O desenvolvimento do componente “R” do sistema RAG móvel cross-platform permitiu avaliar os trade-offs e sinergias entre diferentes paradigmas de busca de informação. Modelos léxicos puramente baseados em termos (BM25) continuam sendo ferramentas poderosas devido à alta fidelidade em termos técnicos específicos e códigos de erro. Em

contrapartida, modelos densos (KNN com embeddings) resolvem o problema de incompatibilidade de vocabulário ao capturar sinônimos e conceitos semelhantes.

Os resultados demonstram que a indexação híbrida baseada em fusão recíproca (RRF) estabelece um meio termo ideal para a construção do RAG, obtendo o maior recall ($R@10 = 0.5685$) e precisão de topo estável. Como trabalhos futuros, recomenda-se investigar técnicas de ajuste fino (*fine-tuning*) do codificador SentenceTransformers utilizando a própria base de artigos científicos coletados para calibrar ainda mais o espaço vetorial e mitigar os ruídos de falsos positivos conceituais.

O vídeo demonstrando o uso prático, a execução dos notebooks e o funcionamento do sistema híbrido de busca está disponível online no YouTube¹.

Referências

- [1] Bird, S., Klein, E., and Loper, E. (2009). *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. O'Reilly Media.
- [2] Cohan, A., Feldman, S., Beltagy, I., McKinnell, A., Lockard, T., and Lo, K. (2020). SPECTER: Document-level Representation Learning using Citation-informed Transformers. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2270–2282.
- [3] Cormack, G. V., Clarke, C. L., and Buettcher, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual bootstrap methods. *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval (SIGIR)*, 580–587.
- [4] Karpukhin, V., Oguz, B., Min, S., Wu, L., Lederman, D., Li, S., Al-Onaizan, Y., Liang, D., Xiong, J., and Yih, W. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 6769–6781.
- [5] Lewis, P., Perez, E., Piktus, A., Petroni, F., Lewis, M., Riedel, S., and Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459–9474.
- [6] Luan, Y., Jacob, K., Devlin, J., and Chang, M. W. (2021). Sparse, Dense, and Hybrid Information Retrieval. *Transactions of the Association for Computational Linguistics (TACL)*, 9, 615–628.
- [7] Manning, C. D., Raghavan, P., and Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
- [8] Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 3982–3992.
- [9] Robertson, S. and Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.

¹Link de acesso ao vídeo explicativo: <https://youtu.be/WXqr7ucUfEw>